

Microcontroller 1 (Arduino)

```
#include <Servo.h>

// ===== BUTTONS =====
#define WASH_BTN    8
#define RINSE_BTN   9
#define SPIN_BTN    10
#define ESTOP_BTN   11

// ===== LEDS =====
#define GREEN_LED    2
#define YELLOW_LED   3
#define RED_LED      4

// ===== BUZZER =====
#define BUZZER 6

// ===== SERVO =====
#define SERVO_PIN    13
#define SERVO_OPEN    0
#define SERVO_CLOSE  90

// ===== L298N MOTOR =====
#define ENA  39    // PWM
#define IN1  37
#define IN2  35

Servo drainServo;
bool emergencyActive = false;

// ===== SEND STATUS =====
void sendStatus(const char* status) {
    Serial2.println(status); // TX2 → RX0 on ESP32 (Telegram)
    Serial1.println(status); // TX1 → RX1 on LCD Mega
}

// ===== MOTOR =====
void motorCW(int pwm) {
```

```

    digitalWrite(IN1, HIGH);
    digitalWrite(IN2, LOW);
    analogWrite(ENA, pwm);
}

void motorCCW(int pwm) {
    digitalWrite(IN1, LOW);
    digitalWrite(IN2, HIGH);
    analogWrite(ENA, pwm);
}

void motorStop() {
    digitalWrite(IN1, LOW);
    digitalWrite(IN2, LOW);
    analogWrite(ENA, 0);
}

// ===== BUZZER =====
void beepOnce() {
    digitalWrite(BUZZER, HIGH);
    delay(100);
    digitalWrite(BUZZER, LOW);
}

void beepEnd() {
    for (int i = 0; i < 3; i++) {
        digitalWrite(BUZZER, HIGH);
        delay(150);
        digitalWrite(BUZZER, LOW);
        delay(150);
    }
}

// ===== SAFE DELAY WITH EMERGENCY =====
bool safeDelay(unsigned long duration) {
    unsigned long start = millis();
    while (millis() - start < duration) {
        if (digitalRead(ESTOP_BTN) == LOW) {
            emergencyActive = true;
            motorStop();

```

```

        drainServo.write(SERVO_CLOSE);
        digitalWrite(GREEN_LED, LOW);
        digitalWrite(YELLOW_LED, LOW);
        digitalWrite(RED_LED, HIGH);
        sendStatus("EMERGENCY");
        return true;
    }
}
return false;
}

// ===== AGITATION =====
bool washAgitate(unsigned long totalTime, int pwm) {
    unsigned long start = millis();
    while (millis() - start < totalTime) {
        if (emergencyActive) return true;

        motorCW(pwm);

        if (safeDelay(2000)) return true;

        motorStop();

        if (safeDelay(500)) return true;

        motorCCW(pwm);

        if (safeDelay(2000)) return true;

        motorStop();

        if (safeDelay(500)) return true;
    }
    return false;
}

// ===== SPIN =====
bool spinMode(unsigned long totalTime, int pwm) {
    motorCW(pwm);
    sendStatus("SPINNING");

```

```

    if (safeDelay(totalTime)) {
        motorStop();
        sendStatus("STOP");
        return true;
    }
    motorStop();
    sendStatus("STOP");
    return false;
}

// ===== DRAIN =====
bool drainProcess() {
    motorStop();
    digitalWrite(GREEN_LED, LOW);
    digitalWrite(YELLOW_LED, HIGH);
    drainServo.write(SERVO_OPEN);
    sendStatus("DRAIN");

    if (safeDelay(5000)) { // yellow LED duration
        drainServo.write(SERVO_CLOSE);
        return true;
    }

    drainServo.write(SERVO_CLOSE);
    digitalWrite(YELLOW_LED, LOW);
    return false;
}

// ===== RUN PROCESS =====
void runProcess(const char* name) {
    if (emergencyActive) return;

    beepOnce();
    sendStatus(name);

    digitalWrite(RED_LED, LOW);
    digitalWrite(GREEN_LED, HIGH);
    digitalWrite(YELLOW_LED, LOW);

    bool stopped = false;

```

```

if (strcmp(name, "WASH") == 0) {
    stopped = washAgitate(10000, 140);
    if (!stopped) stopped = drainProcess();
} else if (strcmp(name, "RINSE") == 0) {
    stopped = washAgitate(8000, 160);
    if (!stopped) stopped = drainProcess();
} else if (strcmp(name, "SPIN") == 0) {
    stopped = spinMode(6000, 220);
}

if (!stopped) {
    sendStatus("DONE");
    digitalWrite(RED_LED, HIGH);
    beepEnd();
} else {
    motorStop();
    drainServo.write(SERVO_CLOSE);
    digitalWrite(GREEN_LED, LOW);
    digitalWrite(YELLOW_LED, LOW);
    digitalWrite(RED_LED, HIGH);
    sendStatus("EMERGENCY");
    while (digitalRead(ESTOP_BTN) == LOW);
    emergencyActive = false;
    sendStatus("READY");
}

while (
    digitalRead(WASH_BTN) == LOW ||
    digitalRead(RINSE_BTN) == LOW ||
    digitalRead(SPIN_BTN) == LOW
);
}

// ===== SETUP =====
void setup() {
    Serial2.begin(9600); // TX2 → RX2 on ESP32 (Telegram)
    Serial1.begin(9600); // TX1 → RX1 on LCD Mega

    pinMode(WASH_BTN, INPUT_PULLUP);

```

```

pinMode(RINSE_BTN, INPUT_PULLUP);
pinMode(SPIN_BTN, INPUT_PULLUP);
pinMode(ESTOP_BTN, INPUT_PULLUP);

pinMode(GREEN_LED, OUTPUT);
pinMode(YELLOW_LED, OUTPUT);
pinMode(RED_LED, OUTPUT);
pinMode(BUZZER, OUTPUT);

pinMode(ENA, OUTPUT);
pinMode(IN1, OUTPUT);
pinMode(IN2, OUTPUT);

drainServo.attach(SERVO_PIN);
drainServo.write(SERVO_CLOSE);

motorStop();
digitalWrite(RED_LED, HIGH);
sendStatus("READY");
}

// ===== LOOP =====
void loop() {
    if (digitalRead(ESTOP_BTN) == LOW && !emergencyActive) {
        emergencyActive = true;
        motorStop();
        drainServo.write(SERVO_CLOSE);
        digitalWrite(GREEN_LED, LOW);
        digitalWrite(YELLOW_LED, LOW);
        digitalWrite(RED_LED, HIGH);
        sendStatus("EMERGENCY");
        while (digitalRead(ESTOP_BTN) == LOW);
        emergencyActive = false;
        sendStatus("READY");
    }

    if (!emergencyActive) {
        if (digitalRead(WASH_BTN) == LOW) {
            runProcess("WASH");
        }
    }
}

```

```

    if (digitalRead(RINSE_BTN) == LOW) {
        runProcess("RINSE");
    }
    if (digitalRead(SPIN_BTN) == LOW) {
        runProcess("SPIN");
    }
}
}

```

Microcontroller 2 (Arduino)

```

#include <Wire.h>
#include <LiquidCrystal_I2C.h>

LiquidCrystal_I2C lcd(0x27, 16, 2);

void setup() {
    Serial1.begin(9600); // RX1 on Mega
    lcd.init();
    lcd.backlight();
    lcd.clear();
    lcd.print("READY");
}

void loop() {
    if (Serial1.available()) {
        String msg = Serial1.readStringUntil('\n');
        msg.trim();
        lcd.clear();
        lcd.setCursor(0,0);
        lcd.print(msg);
    }
}

```

Microcontroller 3 (ESP 32)

```
#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <UniversalTelegramBot.h>

// ===== WiFi =====
const char* ssid = "irdikun";
const char* password = "19101991";

// ===== Telegram Bot =====
#define BOT_TOKEN "8187649563:AAFq1zp2Xcn6qS1mxCwp51qJOAibkssm--E"
#define CHAT_ID "992898239"

WiFiClientSecure client;
UniversalTelegramBot bot(BOT_TOKEN, client);

// ===== Serial from Mega =====
// Using Serial2 on ESP32 (RX2 = GPIO16, TX2 = GPIO17)
HardwareSerial MegaSerial(2);
#define MEGA_RX 16 // RX2
#define MEGA_TX 17 // TX2 (not used for receiving)

String lastStatus = "";

void setup() {
    Serial.begin(9600); // Debug serial
    MegaSerial.begin(9600, SERIAL_8N1, 16, 17);

    // Connect to WiFi
    WiFi.begin(ssid, password);
    Serial.print("Connecting WiFi");
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }
    Serial.println("\nWiFi connected");

    client.setInsecure(); // Accept all certificates
}
```



```
void loop() {  
  // Read status from Mega  
  if (MegaSerial.available()) {  
    String status = MegaSerial.readStringUntil('\n');  
    status.trim();  
  
    if (status.length() > 0 && status != lastStatus) {  
      lastStatus = status;  
      Serial.println("Received: " + status);  
  
      // Send message to Telegram  
      bot.sendMessage(CHAT_ID, "Washing Machine Status: " + status, "");  
    }  
  }  
  
  delay(100);  
}
```